UNIVERSIDADE PAULISTA

CIÊNCIA DA COMPUTAÇÃO

VIAAGORA: APLICAÇÃO DE SISTEMA DISTRIBUÍDO PARA DISPOSITIVO MÓVEL
TEMPO DE ESPERA DAS BALSAS DA BAIXADA SANTISTA

Bruno Nascimento de P. Silva – RA G793030

SANTOS-SP

2026

UNIVERSIDADE PAULISTA

CIÊNCIA DA COMPUTAÇÃO

VIAAGORA: APLICAÇÃO DE SISTEMA DISTRIBUÍDO PARA DISPOSITIVO MÓVEL
TEMPO DE ESPERA DAS BALSAS DA BAIXADA SANTISTA

Atividade prática supervisionada apresentada ao curso de Ciência da Computação, disciplina de Desenvolvimento de Sistemas Distribuídos, para a obtenção de nota na mesma.

Bruno Nascimento de P. Silva – RA G793030

SANTOS-SP

2026

ÍNDICE

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO 4
2 INTRODUÇÃO 5
2.1 Interdisciplinaridade e formação 6
3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS 7
3.1 Plataforma Android e linguagem Kotlin 7
3.2 Sistemas distribuídos 7
3.3 Comunicação: HTTP, WebSocket e AMQP 8
3.4 Persistência e o padrão outbox 8
3.5 Servidor: Node.js, Fastify e Zod 9
3.6 Contêineres, Docker Compose e Nginx 9
3.7 Coleta de dados da web 9
3.8 Tempo e sincronização 9
3.9 Mapas e geolocalização 10
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO 11
4.1 Metodologia 11
4.2 Requisitos 12
4.3 Testes 13
4.4 Tolerância a falhas 13
4.5 Segurança e privacidade 13
4.6 Casos de uso 14
4.7 Cronograma 14
4.8 Riscos e mitigação 14
4.9 Publicação 14
5 PROJETO DO PROGRAMA 16
5.1 Arquitetura 16
5.2 O que torna o ViaAgora um sistema distribuído 16
5.3 Módulos 17
5.4 Modelo de dados 17
5.5 Caminho de um relato 18
6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA 19
6.1 Varredura do painel oficial 19
6.2 Relato e evento na mesma transação 19
6.3 Worker: publicação com confirmação 20
6.4 Cada réplica recebe todos os eventos 21
6.5 Coletor com espera crescente 22
6.6 Canal ao vivo no aplicativo 22
6.7 Balanceamento no Nginx 23
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO 24
8 CONCLUSÃO 27
BIBLIOGRAFIA 28
FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS 29

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

O objetivo deste trabalho é desenvolver o **ViaAgora**, uma aplicação de sistema distribuído para dispositivo móvel que informa, em tempo real, o tempo de espera das balsas da Baixada Santista, nas travessias entre Santos e Guarujá, entre Santos e Vicente de Carvalho e entre Bertioga e Guarujá. O tema da APS pede o gerenciamento de informações ambientais urbanas, e o enunciado cita entre os exemplos o trânsito e o transporte público. As travessias se encaixam nos dois: são um serviço público de transporte e, quando a fila cresce, formam congestionamento nas avenidas de acesso, com veículos parados e motores ligados.

A motivação é prática. Quem mora na região sabe que a espera muda muito ao longo do dia e com o mar: maré baixa, neblina ou mar agitado reduzem as embarcações e chegam a parar a travessia. O Governo do Estado publica um painel com a espera atual, mas ele não guarda histórico, não mostra o que as pessoas na fila estão vendo e não avisa ninguém quando algo muda. O aplicativo junta essas duas fontes, a oficial e a da comunidade, e entrega a informação no celular assim que ela muda.

O segundo objetivo é acadêmico: aplicar, num problema real, os conceitos de sistemas distribuídos estudados na disciplina. O servidor foi dividido em processos independentes (duas réplicas da API, um coletor, um worker, um broker de mensagens e um banco de dados) que se comunicam por HTTP, WebSocket e AMQP e continuam funcionando quando uma das partes falha.

Os objetivos específicos foram:

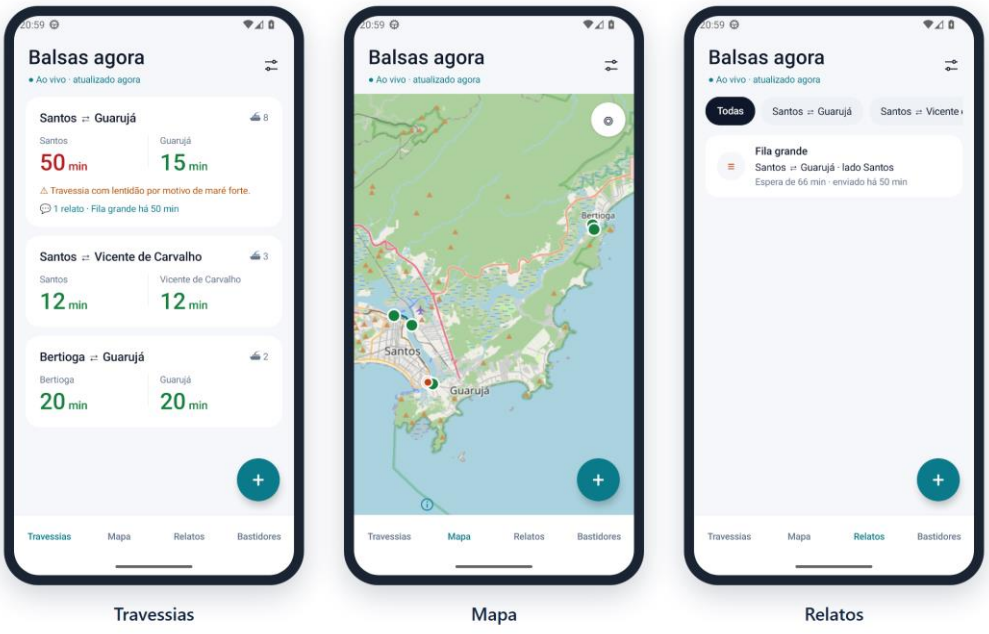
- coletar automaticamente os dados do painel oficial e guardar o histórico de cada travessia;
- receber os relatos da comunidade e entregá-los a todos os celulares em poucos segundos;
- aplicar replicação, mensageria e tolerância a falhas, e mostrá-las na própria interface (aba Bastidores);
- permitir que qualquer pessoa execute o sistema completo no próprio computador com um único comando.

2 INTRODUÇÃO

As balsas da Baixada Santista são operadas pelo Departamento Hidroviário do Estado de São Paulo e transportam, todos os dias, moradores, trabalhadores e turistas entre Santos, Guarujá, Vicente de Carvalho e Bertioga. O tempo de espera é publicado no site das Travessias Litorâneas da SEMIL-SP, mas o painel é montado no navegador por um script e não oferece uma API pública documentada.

O ViaAgora resolve esse problema em três partes. Um **coletor** no servidor lê o painel oficial a cada minuto, como um navegador faria, e grava cada leitura no banco de dados. Os usuários podem enviar **relatos** pelo aplicativo (fila grande, embarque lento, travessia parada, neblina ou acidente), e o servidor calcula a mediana das esperas informadas nos últimos 45 minutos. Por fim, toda mudança é **avisada ao vivo** a todos os celulares conectados, passando por um broker de mensagens que distribui o aviso entre as réplicas da API.

Figura 1 – Telas principais do aplicativo: travessias, mapa e relatos



Fonte: elaborado pelo autor (2026).

O sistema está publicado numa VPS, no endereço sedda.com.br, e o aplicativo abre conectado a ele sem nenhuma configuração. O mesmo conjunto de processos pode ser executado no computador de qualquer pessoa com um único comando do Docker, o que permite estudar e testar o sistema distribuído localmente.

2.1 Interdisciplinaridade e formação

O projeto reuniu conteúdos de várias disciplinas do curso. De **redes de computadores** vieram os protocolos HTTP, WebSocket e AMQP e o papel do balanceador de carga. De **banco de dados**, as transações, as restrições de integridade e as consultas agregadas do histórico. De **engenharia de software**, a divisão em módulos, os testes automatizados e o controle de versões. De **programação para dispositivos móveis**, o aplicativo Android em Kotlin. A **estatística** aparece na mediana dos relatos e nas médias por hora do melhor horário, e o tema ambiental urbano liga tudo isso a um problema de mobilidade da cidade onde o trabalho foi feito.

Para a formação, o principal ganho foi ver na prática que um sistema distribuído é, antes de tudo, um sistema que precisa continuar correto quando as partes falham. Decisões como gravar o evento na mesma transação do dado (outbox) ou fazer o aplicativo buscar o estado completo depois de cada aviso só fazem sentido quando se testa o que acontece ao desligar o broker ou uma réplica no meio do uso.

3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS

3.1 Plataforma Android e linguagem Kotlin

O Android é o sistema operacional móvel mais usado no Brasil, o que justificou a escolha da plataforma. A aplicação foi escrita em **Kotlin**, a linguagem recomendada pelo Google para Android, que combina tipos seguros contra valores nulos, funções de ordem superior e classes de dados com interoperabilidade total com as bibliotecas Java. O aplicativo segue o ciclo de vida da Activity (criação, início, pausa e destruição) para abrir e fechar conexões no momento certo e não gastar bateria em segundo plano. Para o mapa foi usada a biblioteca **MapLibre Native** com os dados abertos do **OpenStreetMap**, de onde também foram tiradas as rotas reais das balsas.

3.2 Sistemas distribuídos

Segundo Tanenbaum e Van Steen (2007), um sistema distribuído é um conjunto de computadores independentes que se apresenta aos usuários como um sistema único e coerente. Coulouris et al. (2013) destacam as consequências disso: os componentes executam de forma **concorrente**, **não há relógio global** e cada parte **falha de forma independente**, de modo que a comunicação acontece apenas por troca de mensagens. Um dos objetivos de projeto é a **transparência**: esconder do usuário que o serviço está espalhado por vários processos. No ViaAgora, o celular conversa com um único endereço (transparência de acesso e de localização), não sabe qual réplica o atende (transparência de replicação) e continua funcionando quando uma réplica ou o broker caem (transparência de falha).

Tabela 1 – Conceitos de sistemas distribuídos aplicados no projeto

Conceito	Como aparece no ViaAgora
Replicação	Duas réplicas idênticas da API atendem os celulares; se uma cair, a outra continua.
Balanceamento de carga	O Nginx recebe todos os acessos e os reparte entre as réplicas.
Publish/subscribe	O RabbitMQ entrega uma cópia de cada evento a todas as réplicas (exchange fanout).
Consistência entre banco e mensagens	Padrão transactional outbox: o evento é gravado na mesma transação do dado.
Idempotência	Reenviar o mesmo relato (Idempotency-Key) não cria um relato duplicado.
Tolerância a falhas	Reconexão, ressincronização, backoff do coletor e dado marcado como atrasado.

Conceito	Como aparece no ViaAgora
Monitoramento	Cada processo grava um sinal de vida (heartbeat) a cada 5 segundos.

Fonte: elaborado pelo autor (2026).

3.3 Comunicação: HTTP, WebSocket e AMQP

O aplicativo conversa com o servidor por **HTTP**, no modelo de pedido e resposta, usando JSON. Para receber avisos sem ficar perguntando, abre também um **WebSocket** (FETTE; MELNIKOV, 2011), uma conexão que permanece aberta e por onde o servidor envia mensagens a qualquer momento. Entre os processos do servidor, os eventos trafegam pelo protocolo **AMQP** até o RabbitMQ, que guarda as mensagens e as entrega com confirmação: um evento só é considerado publicado depois que o broker confirma o recebimento. É a diferença entre comunicação **síncrona** (o celular espera a resposta da API) e **assíncrona** (o worker publica e cada réplica recebe quando puder, sem que um espere o outro).

No RabbitMQ, o produtor publica numa **exchange**, e é ela que decide para quais filas a mensagem vai. Uma fila única com dois consumidores divide as mensagens entre eles; por isso o projeto usa uma exchange do tipo **fanout**, que entrega uma cópia a cada fila ligada a ela (Figura 2). Cada réplica da API tem a sua fila e, assim, todas ficam sabendo de todos os eventos.

Figura 2 – Fila única (divide as mensagens) e exchange fanout (copia para todos)



Fonte: elaborado pelo autor (2026).

3.4 Persistência e o padrão outbox

O **PostgreSQL** é a fonte da verdade do sistema. Toda escrita acontece dentro de uma transação, que garante atomicidade: ou todas as operações são gravadas, ou nenhuma. O problema clássico é gravar um dado e avisar os outros processos: se o aviso for enviado depois da gravação e o broker estiver fora do ar, o aviso se perde. O padrão **transactional outbox** (RICHARDSON, 2018) resolve isso gravando o

evento numa tabela do próprio banco, na mesma transação do dado. Um processo separado, o worker, lê essa tabela e publica os eventos pendentes. O resultado é uma **consistência eventual** (KLEPPMANN, 2017): por alguns milissegundos as réplicas ainda não sabem do relato, mas todas acabam sabendo, e nada se perde no caminho.

3.5 Servidor: Node.js, Fastify e Zod

O servidor foi escrito em **Node.js 22**, cujo modelo assíncrono atende muitas conexões abertas (os WebSockets) com pouco consumo de memória. As rotas usam o framework **Fastify**, e todos os dados recebidos são validados com a biblioteca **Zod** antes de chegar ao banco. A mesma imagem de código roda em três papéis (API, worker e coletor), escolhidos pelo comando de inicialização.

3.6 Contêineres, Docker Compose e Nginx

Cada processo roda num contêiner **Docker**, isolado e com as mesmas dependências em qualquer máquina. O **Docker Compose** descreve os sete serviços num único arquivo e os inicia na ordem certa, esperando o banco e o broker estarem saudáveis antes de subir as APIs; para o sistema, cada contêiner se comporta como uma máquina separada da rede. Na frente de tudo fica o **Nginx**, que entrega o site do projeto e atua como proxy reverso e balanceador de carga para as duas réplicas, inclusive para as conexões WebSocket.

3.7 Coleta de dados da web

Quando a informação só existe numa página, ela pode ser obtida por **web scraping**: o programa baixa a página como um navegador faria e extrai os dados. No painel da SEMIL os números não estão no HTML; um script do site busca um serviço JSON e desenha o painel. O coletor do ViaAgora refaz esse caminho: lê a página, encontra o script, extrai o endereço do serviço e consulta o JSON. Como a fonte não é uma API pública, o coletor se identifica, consulta no máximo uma vez por minuto a partir de um único processo e espera mais quando a fonte falha.

3.8 Tempo e sincronização

Num sistema distribuído cada máquina tem o seu próprio relógio, e eles nunca estão perfeitamente iguais. Tanenbaum e Van Steen (2007) mostram que, por isso, não se pode confiar na hora de máquinas diferentes para decidir a ordem dos

acontecimentos: é preciso uma referência comum ou um relógio lógico. No ViaAgora, a ordem dos eventos é dada pelo banco de dados: a criação, a publicação e a entrega de cada evento são registradas com o relógio do PostgreSQL, e a aba Bastidores calcula os tempos a partir dessa única referência. O sinal de vida dos processos segue a mesma ideia: um processo é considerado fora do ar quando o banco não recebe notícias dele há mais de 15 segundos.

3.9 Mapas e geolocalização

O mapa usa a biblioteca MapLibre Native, de código aberto, com as camadas do OpenStreetMap. Os terminais e as rotas das balsas vieram das linhas de travessia cadastradas no próprio OpenStreetMap, o que coloca cada marcador exatamente sobre o terminal real. A localização do celular é opcional: o usuário só concede a permissão quando toca no botão de posição, e o relato pode ser marcado em qualquer ponto do mapa.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

4.1 Metodologia

O desenvolvimento foi **incremental**: o sistema cresceu em etapas curtas, e cada etapa só terminava com o que tinha sido feito testado de forma automática e, depois, com o sistema rodando. A ordem das etapas (Tabela 2) seguiu o caminho do dado: primeiro a fonte oficial, depois o banco, a API, a mensageria entre as réplicas e, por último, o aplicativo e a publicação.

Tabela 2 – Etapas do desenvolvimento

Etapa	Entrega
1. Estudo da fonte	Análise do painel da SEMIL-SP e descoberta do serviço que alimenta o painel.
2. Coletor e banco	Varredura do site, normalização dos dados e gravação das leituras.
3. API e relatos	Rotas HTTP, participação anônima, idempotência e outbox.
4. Mensageria	Worker, RabbitMQ com exchange fanout e WebSocket nas duas réplicas.
5. Aplicativo	Telas de travessias, mapa, relatos, melhor horário e Bastidores.
6. Falhas e publicação	Experimentos derrubando broker e réplicas; VPS com HTTPS e ambiente local.

Fonte: elaborado pelo autor (2026).

O código do servidor, do site e a documentação foram editados no **Visual Studio Code**; o aplicativo foi desenvolvido no **Android Studio**, que traz o SDK do Android, o compilador Gradle, o emulador usado nos testes e o Logcat para acompanhar os erros do aparelho. O versionamento foi feito com **Git**, e o código está publicado no **GitHub**. As figuras e diagramas foram elaborados pelo autor, e as telas foram capturadas do aplicativo em execução.

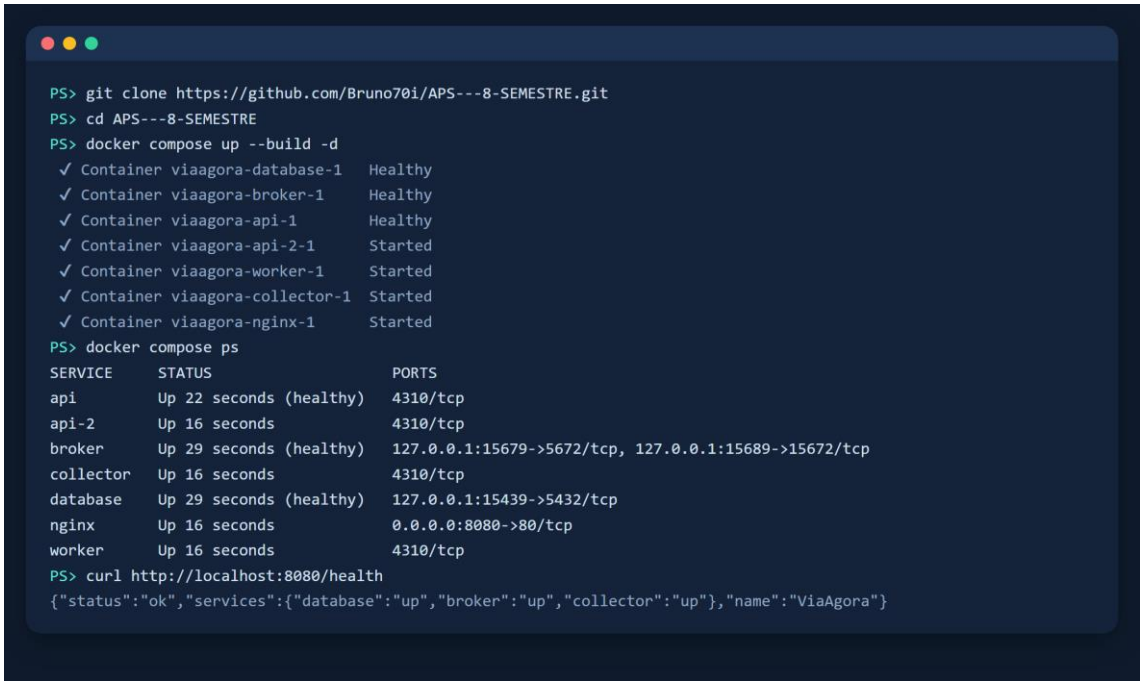
Para construir e observar o sistema distribuído, foram usadas ferramentas específicas:

- **Docker Desktop e Docker Compose**: simulam, num único computador, várias máquinas ligadas em rede, cada uma com o seu processo;
- **Nginx**: faz o balanceamento entre as réplicas e permite testar o que acontece quando uma delas cai;
- **RabbitMQ** com o seu **painel de gerenciamento** (porta 15689 no ambiente local), onde aparecem a exchange, as filas de cada réplica e as mensagens;
- a rota **/v1/system** e a aba **Bastidores**, criadas no próprio projeto para ver os sinais de vida dos processos e o caminho de cada evento;

- os comandos `docker compose logs`, `stop` e `start`, usados para acompanhar os processos e provocar falhas de propósito.

O servidor nunca é instalado diretamente na máquina: tudo roda em contêineres. Com isso, o ambiente de quem baixa o projeto é igual ao da VPS, e o sistema inteiro sobe com um único comando (Figura 3).

Figura 3 – Sistema completo iniciado com o Docker Compose (saída real do terminal)



Fonte: elaborado pelo autor (2026).

4.2 Requisitos

Tabela 3 – Requisitos funcionais

Código	Requisito
RF01	Mostrar a espera de cada lado das três travessias, as embarcações e os avisos oficiais, e indicar quando a travessia está parada.
RF02	Permitir enviar relatos (fila, embarque lento, parada, clima, acidente) e mostrar os relatos de todos os usuários.
RF03	Mostrar os terminais e as rotas das balsas num mapa e calcular o melhor horário com o histórico de 7 dias.
RF04	Avisar os celulares conectados assim que um dado mudar, sem atualização manual.
RF05	Mostrar, na aba Bastidores, os processos do sistema e o caminho de cada evento.

Fonte: elaborado pelo autor (2026).

Os requisitos não funcionais são justamente os de um sistema distribuído: a queda de uma réplica não pode interromper o serviço (**disponibilidade**); nenhum relato aceito pode ser perdido, nem o seu aviso (**consistência**); a fonte oficial é consultada por um único coletor (**respeito à fonte**); o usuário não faz cadastro

(**privacidade**); e o servidor inteiro sobe com um comando em qualquer computador (**portabilidade**).

4.3 Testes

O servidor tem **23 testes automatizados** que rodam com PostgreSQL e RabbitMQ reais, num ambiente separado (`compose.test.yaml`). Eles cobrem a validação dos relatos, a idempotência, a gravação na outbox, a leitura do painel oficial, a entrega dos eventos às duas réplicas, a recuperação depois da queda do broker e a rejeição de mensagens WebSocket grandes demais. O aplicativo tem **12 testes** para as regras de domínio, e a compilação passa pelo Android Lint sem erros.

4.4 Tolerância a falhas

Cada parte do sistema foi pensada para falhar sem levar as outras junto. A Figura 4 resume o comportamento em cada caso; todos foram testados derrubando os contêineres com o sistema em uso.

Figura 4 – Comportamento do sistema quando cada parte falha



Fonte: elaborado pelo autor (2026).

4.5 Segurança e privacidade

O aplicativo não pede cadastro. Na primeira abertura ele recebe um token aleatório, e o servidor guarda apenas o hash SHA-256 desse token, o suficiente para saber quem pode encerrar cada relato. Todo dado recebido é validado (tipos, limites e tamanho), as consultas ao banco usam parâmetros, o que evita injeção de SQL, e o tráfego público passa por HTTPS. As senhas do banco e do broker ficam só no servidor, fora do código e do APK.

4.6 Casos de uso

Tabela 4 – Perfis de usuário e casos de uso

Perfil	Situação	O que o aplicativo oferece
Morador em casa	Decide a hora de sair para atravessar	Espera atual de cada lado e melhor horário do dia
Motorista na fila	Vê a fila maior do que o painel indica	Envia um relato com a espera real em poucos toques
Turista	Não conhece os terminais	Mapa com os terminais, as rotas e a espera colorida
Estudante ou professor	Quer ver o sistema distribuído funcionando	Aba Bastidores e ambiente local com Docker

Fonte: elaborado pelo autor (2026).

4.7 Cronograma

O trabalho foi feito entre agosto e outubro de 2026. Agosto foi dedicado ao planejamento, à pesquisa da fonte de dados e ao desenho da arquitetura distribuída; setembro, ao servidor (banco, coletor, API e mensageria com RabbitMQ e outbox); e outubro, ao aplicativo Android, à aba Bastidores, aos testes de falha, à publicação na VPS e à documentação. A maior parte do tempo foi para a mensageria e para o aplicativo, as partes em que os processos precisam conversar entre si.

4.8 Riscos e mitigação

Tabela 5 – Riscos identificados no planejamento

Risco	Mitigação adotada
O site oficial mudar ou sair do ar	Descoberta automática do endereço, último dado mantido e marcado como atrasado
Perda de avisos com o broker fora do ar	Outbox transacional e publicação com confirmação
Relato enviado duas vezes por falha de rede	Chave de idempotência por envio
Queda de uma réplica da API	Duas réplicas atrás do Nginx, com troca automática
Celular perde o canal ao vivo	Reconexão, ressincronização e nova busca a cada 30 segundos

Fonte: elaborado pelo autor (2026).

4.9 Publicação

O sistema foi publicado numa VPS Linux com o domínio sedda.com.br e certificado HTTPS. O mesmo Nginx que reparte os celulares entre as réplicas entrega o site do projeto (Figura 5), que explica a arquitetura de forma interativa e oferece o APK para download e as instruções para rodar o sistema localmente.

Figura 5 – Site do projeto em sedda.com.br: diagrama interativo da arquitetura



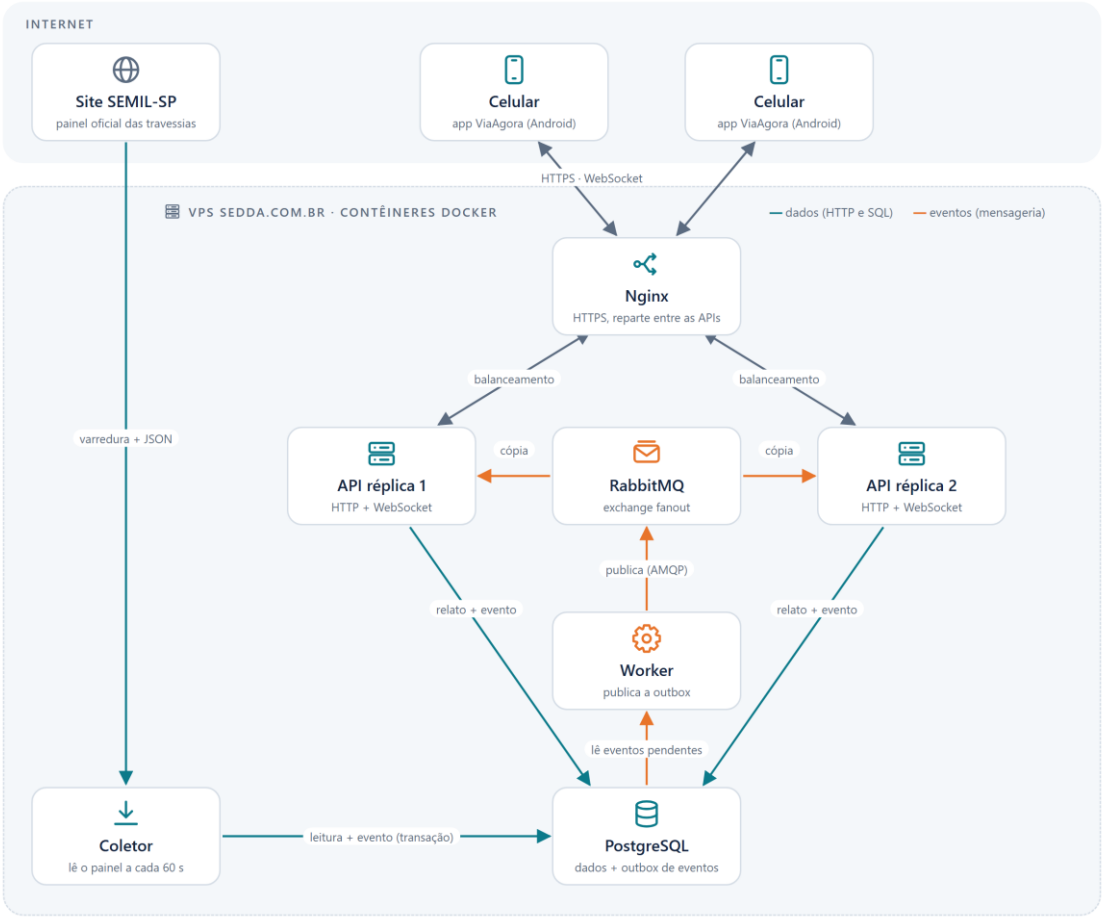
Fonte: elaborado pelo autor (2026).

5 PROJETO DO PROGRAMA

5.1 Arquitetura

A Figura 6 mostra a arquitetura. Na internet ficam os celulares e o site oficial; na VPS, sete contêineres. O Nginx é a única porta de entrada e reparte os celulares entre as duas réplicas da API. As linhas verdes são dados (HTTP e SQL) e as laranjas, eventos (mensageria).

Figura 6 – Arquitetura distribuída do ViaAgora



Fonte: elaborado pelo autor (2026).

5.2 O que torna o ViaAgora um sistema distribuído

As características que Coulouris et al. (2013) apontam nos sistemas distribuídos aparecem diretamente no projeto:

- **Processos independentes**: API, worker, coletor, banco e broker rodam em contêineres separados e só se comunicam pela rede; nenhum deles lê a memória do outro.

- **Concorrência**: dois celulares podem enviar relatos ao mesmo tempo para réplicas diferentes, e o banco resolve o acesso simultâneo; dois workers nunca publicam o mesmo evento, graças ao `FOR UPDATE SKIP LOCKED`.
- **Sem relógio global**: cada contêiner tem o seu relógio. Por isso os horários do caminho de um evento (criação, publicação e entrega) são medidos pelo relógio do PostgreSQL, a referência comum a todos.
- **Falhas independentes**: o broker pode cair sem levar a API junto, e uma réplica pode cair sem levar a outra; o sistema percebe pelo sinal de vida e se recupera sozinho.
- **Escalabilidade horizontal**: para atender mais celulares basta acrescentar réplicas da API no Compose e no Nginx, sem mudar o código, porque elas não guardam estado próprio.

5.3 Módulos

Tabela 6 – Módulos principais

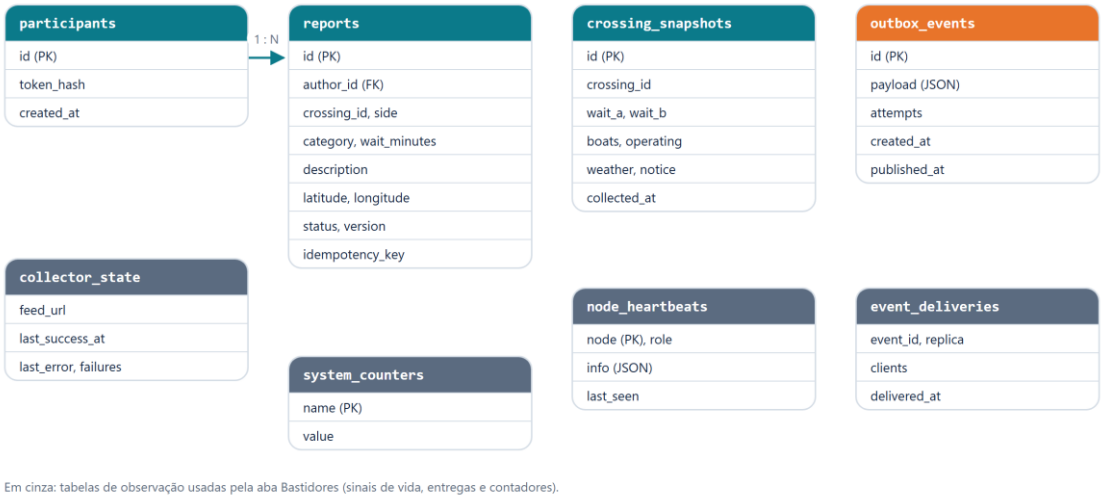
Módulo	Responsabilidade
`server/src/app.js` (`montarApp`)	Rotas HTTP da API e validação das entradas.
`server/src/domain.js` (`salvarRelato`)	Regras de negócio, transações e gravação na outbox.
`server/src/broker.js` (`publicarLote`)	Publicação no RabbitMQ e WebSocket de cada réplica.
`server/src/crossings.js` (`lerPainel`)	Catálogo das travessias e leitura do painel oficial.
`MainActivity.kt`, `RepositorioRemoto.kt`	Telas do app, chamadas HTTP e canal ao vivo.
`PaginaBastidores.kt`, `DiagramaRede.kt`	Aba Bastidores: diagrama ao vivo e caminho dos eventos.

Fonte: elaborado pelo autor (2026).

5.4 Modelo de dados

O banco tem quatro tabelas principais e quatro de observação (Figura 7). A tabela `outbox_events` é o elo entre o banco e a mensageria: cada linha é um evento que ainda precisa ser publicado (`published_at` vazio) ou que já foi.

Figura 7 – Tabelas do PostgreSQL



Fonte: elaborado pelo autor (2026).

5.5 Caminho de um relato

A Figura 8 mostra o caminho completo de um relato, do celular que envia até os celulares que recebem o aviso, pelas rotas `POST /v1/reports` e `WS /v1/events` (a lista completa de rotas está na Documentação). O aplicativo não confia no conteúdo do aviso: ao recebê-lo, busca o estado atual na API. Assim, mesmo que um aviso chegue repetido ou fora de ordem, a tela sempre mostra o que está gravado no banco.

Figura 8 – Caminho de um relato pelo sistema



Fonte: elaborado pelo autor (2026).

6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

As listagens a seguir mostram os trechos que implementam os conceitos distribuídos, com os números de linha dos arquivos do repositório. O código completo está no documento de Documentação, entregue à parte.

6.1 Varredura do painel oficial

O painel da SEMIL é desenhado por um script. O coletor encontra esse script na página e extrai dele o endereço do serviço de dados, ignorando linhas comentadas. Assim, se o governo trocar o endereço, o coletor o descobre sozinho.

Listagem 1 – Descoberta do serviço do painel (server/src/crossings.js, linhas 36 a 64)

```
36 // Página da SEMIL -> <script ...travessia-service...> -> variável getInfo (ignora linhas comentadas).
37 export function encontrarScriptDoPainel(html,base=ENDERECO_SITE){
38   const endereco=html.match(/<script[^>]+src="([^"]*travessia-service[^"]*)"/i)?.[1];
39   return endereco?new URL(endereco,base).toString():null;
40 }
41 export function encontrarEnderecoServico(script){
42   for(const linha of script.split(/\r?\n/)){
43     if(linha.trim().startsWith('//'))continue;
44     const endereco=linha.match(/getInfo\s*=\s*["'](https?:\/\/[^"']+)["']/)?.[1];
45     if(endereco)return endereco.endsWith('/')?endereco:endereco+'/';
46   }
47   return null;
48 }
49
50 // Baixa uma página ou um JSON, desistindo depois de 15 segundos.
51 async function baixar(buscarImpl,endereco,tipo){
52   const resposta=await buscarImpl(endereco,{headers:cabecalhos,signal:AbortSignal.timeout(15000)});
53   if(!resposta.ok)throw new Error(`Fonte respondeu ${resposta.status} em ${new URL(endereco).host}`);
54   return tipo==='json'?resposta.json():resposta.text();
55 }
56 // Refaz o caminho do navegador: página -> script do painel -> endereço do serviço de dados.
57 export async function descobrirEnderecoServico(buscarImpl=fetch){
58   const html=await baixar(buscarImpl,ENDERECO_SITE,'text');
59   const script=encontrarScriptDoPainel(html);
60   if(!script)throw new Error('Script do painel não encontrado na página.');
61   const base=encontrarEnderecoServico(await baixar(buscarImpl,script,'text'));
62   if(!base)throw new Error('Endereço do serviço não encontrado no script.');
63   return base+'PT';
64 }
```

Fonte: código do projeto ViaAgora (2026).

6.2 Relato e evento na mesma transação

A função `salvarRelato` grava o relato e o evento `report.created` na outbox dentro da mesma transação. A restrição única `(author_id, idempotency_key)` faz o reenvio do mesmo relato devolver o relato existente em vez de criar outro.

Listagem 2 – Transação, outbox e idempotência (server/src/domain.js, linhas 22 a 55)

```
22  // Executa "operacao" dentro de uma transação do PostgreSQL (BEGIN ... COMMIT, ou ROLLBACK se der
erro).
23  async function transacao(pool,operacao){
24    const conexao=await pool.connect();
25    try{await conexao.query('BEGIN');const resultado=await operacao(conexao);await
conexao.query('COMMIT');return resultado;}
26    catch(erro){await conexao.query('ROLLBACK');throw erro;}
27    finally{conexao.release();}
28  }
29  // Grava um evento na outbox, na mesma transação do dado. O worker publica depois no RabbitMQ.
30  // "origem" diz quem gerou o evento (api-1, api-2 ou collector), para a aba Bastidores desenhar o
caminho.
31  async function registrarEvento(conexao,dados,origem){
32    const evento={eventId:randomUUID(),...dados,origin:origem};
33    await conexao.query('INSERT INTO outbox_events(id,payload) VALUES($1,$2)',[evento.eventId,evento]);
34  }
35
36  // Salva um relato e o evento "report.created" juntos. A chave de idempotência (única por
participante)
37  // faz o reenvio do mesmo relato devolver o relato existente em vez de criar outro.
38  export async function salvarRelato(pool,autorId,entrada,chave,origem='api'){
39    const hashDoPedido=gerarHash(JSON.stringify(entrada));
40    return transacao(pool,async conexao=>{
41      const inserido=await conexao.query(`INSERT INTO
reports(id,author_id,crossing_id,side,category,wait_minutes,description,latitude,longitude,idempotency_key
,request_hash)
42        VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11) ON CONFLICT(author_id,idempotency_key) DO NOTHING
RETURNING *`,
43
[randomUUID(),autorId,entrada.crossingId,entrada.side,entrada.category,entrada.waitMinutes,entrada.descrip
tion,entrada.latitude,entrada.longitude,chave,hashDoPedido]);
44      if(inserido.rowCount){
45        const relato=paraRelato(inserido.rows[0]);
46        await
registrarEvento(conexao,{type:'report.created',reportId:relato.id,crossingId:relato.crossingId,version:rel
ato.version},origem);
47        return {relato,reenvio:false};
48      }
49      const existente=(await conexao.query('SELECT * FROM reports WHERE author_id=$1 AND
idempotency_key=$2',[autorId,chave])).rows[0];
50      if(existente.request_hash!==hashDoPedido)throw new ErroHttp(409,'Esta chave já foi usada para
outro relato.');
51      // Reenvio do mesmo relato (idempotência): nada é duplicado, só contamos para mostrar na aba
Bastidores.
52      await conexao.query(`INSERT INTO system_counters(name,value) VALUES('idempotent_replays',1) ON
CONFLICT(name) DO UPDATE SET value=system_counters.value+1`);
53      return {relato:paraRelato(existente),reenvio:true};
54    });
55  }
```

Fonte: código do projeto ViaAgora (2026).

6.3 Worker: publicação com confirmação

A função `publicarLote` trava as linhas pendentes com `FOR UPDATE SKIP LOCKED` (dois workers nunca pegam o mesmo evento), publica cada uma num canal com confirmação e só marca `published_at` depois que o RabbitMQ confirma. Se o broker cair, a transação é desfeita e os eventos continuam pendentes.

Listagem 3 – Publicação dos eventos da outbox (server/src/broker.js, linhas 69 a 91)

```
69  // Publica até 25 eventos pendentes da outbox. FOR UPDATE SKIP LOCKED garante que dois workers nunca
peguem o mesmo
70  // evento; o evento só é marcado como publicado depois que o RabbitMQ confirma. Se algo falhar, a
transação é
71  // desfeita e os eventos continuam pendentes para a próxima tentativa.
72  export async function publicarLote(pool,url=urlRabbit()){
73    const conexaoBanco=await pool.connect();let conexao,canal;
74    try{
75      await conexaoBanco.query('BEGIN');
76      const pendentes=await conexaoBanco.query('SELECT id,payload FROM outbox_events WHERE published_at
IS NULL ORDER BY created_at LIMIT 25 FOR UPDATE SKIP LOCKED');
77      if(!pendentes.rowCount){await conexaoBanco.query('COMMIT');return 0;}
78      conexao=await amqp.connect(url,{timeout:2000});conexao.on('error',()=>{});
79      canal=await conexao.createConfirmChannel();canal.on('error',()=>{});
80      await canal.assertExchange(exchange,'fanout',{durable:true});
81      for(const linha of pendentes.rows){
82        // O consumidor pode receber antes de a confirmação chegar ao worker. Medimos
83        // a saída com o relógio do banco, mas só encerramos a pendência após confirmar.
84        const publicadoEm=(await conexaoBanco.query('SELECT clock_timestamp() AS
sent_at')).rows[0].sent_at;
85        await new
Promise((resolver,rejeitar)=>canal.publish(exchange,'',Buffer.from(JSON.stringify(linha.payload)),{persist
ent:true,messageId:linha.id,contentType:'application/json'},erro=>erro?rejeitar(erro):resolver()));
86        await conexaoBanco.query('UPDATE outbox_events SET published_at=$2,attempts=attempts+1 WHERE
id=$1',[linha.id,publicadoEm]);
87      }
88      await conexaoBanco.query('COMMIT');return pendentes.rowCount;
89    }catch(erro){await conexaoBanco.query('ROLLBACK');throw erro;}
90    finally{await fecharComCalma(canal);await fecharComCalma(conexao);conexaoBanco.release();}
91  }
```

Fonte: código do projeto ViaAgora (2026).

6.4 Cada réplica recebe todos os eventos

Na função `conectar`, cada réplica da API cria uma fila exclusiva ligada à exchange fanout. Por isso todas recebem uma cópia de cada evento e avisam os seus próprios celulares pelo WebSocket. Ao reconectar ao broker, a réplica envia `resync`, e os celulares buscam tudo de novo.

Listagem 4 – Consumo do fanout e aviso aos celulares (server/src/broker.js, linhas 35 a 59)

```
35    async function conectar(){
36      try{
37        conexao=await amqp.connect(url,{timeout:3000});
38        conexao.on('error',()=>{});conexao.on('close',tentarDeNovo);
39        if(encerrando){await fecharComCalma(conexao);return;}
40        canal=await conexao.createChannel();canal.on('error',()=>{});
41        await canal.assertExchange(exchange,'fanout',{durable:true});
42        // Fila exclusiva desta réplica, ligada à exchange fanout: assim TODAS as réplicas recebem cada
evento.
43        const fila=await canal.assertQueue('',{exclusive:true,autoDelete:true});
44        await canal.bindQueue(fila.queue,exchange,'');
45        await canal.consume(fila.queue,mensagem=>{
46          if(!mensagem)return;
47          try{
48            const evento=JSON.parse(mensagem.content.toString());
49            // Cada réplica carimba quem entregou e quando, e registra a entrega para a aba Bastidores.
50            avisarCelulares({...evento,via:replica,deliveredAt:new
Date().toISOString()});canal.ack(mensagem);
51            if(pool&&evento.eventId)pool.query('INSERT INTO event_deliveries(event_id,replica,clients)
VALUES($1,$2,$3) ON CONFLICT DO NOTHING',
52              [evento.eventId,replica,servidorWs.clients.size]).catch(()=>{});
53          }catch{canal.nack(mensagem,false,false);}
54        });
55        conectado=true;
56        // Reconectou ao broker: pede aos celulares que busquem tudo de novo (algum evento pode ter
passado).
57        avisarCelulares({type:'resync',connected:true});
58      }catch{await fecharComCalma(conexao);tentarDeNovo();}
59    }
```

Fonte: código do projeto ViaAgora (2026).

6.5 Coletor com espera crescente

Em caso de falha, o coletor dobra o intervalo entre as tentativas, até 10 minutos, para não sobrecarregar a fonte oficial. O último dado bom continua disponível na API.

Listagem 5 – Laço do coletor (server/src/collector.js, linhas 13 a 21)

```
13  while(!encerrando){
14    try{const {gravadas,mudancas}=await
coletor.executar();falhas=0;ultimaDeuCerto=true;console.log(`Coleta: ${gravadas} travessias, ${mudancas}
com mudança.`);}
15    catch(erro){falhas++;ultimaDeuCerto=false;console.error('Coleta falhou; último dado
preservado:',erro.message);}
16    ultimaColeta=new Date().toISOString();
17    if(++coletas%60===0)await pool.query("DELETE FROM crossing_snapshots WHERE collected_at<now()-
interval '30 days'").catch(()=>{});
18    // Em falha, espera mais a cada tentativa (até 10 min) para não sobrecarregar a fonte oficial.
19    const espera=Math.min(intervalo*2**Math.min(falhas,4),600_000);
20    for(let esperado=0;esperado<espera&&!encerrando;esperado+=500)await new
Promise(resolver=>setTimeout(resolver,500));
21  }
```

Fonte: código do projeto ViaAgora (2026).

6.6 Canal ao vivo no aplicativo

No Android, a função `conectar` do `RepositorioRemoto` abre o WebSocket com o OkHttp. Cada aviso dispara uma nova busca do estado; se a conexão cair, o aplicativo tenta de novo em 2,5 segundos.

Listagem 6 – WebSocket com reconexão
(android/app/src/main/java/br/com/viaagora/RepositorioRemoto.kt, linhas 91 a 111)

```
91      // aoEvento recebe cada mensagem do WebSocket (tipo, réplica que entregou, se é um evento
rastreável) para a aba Bastidores.
92      fun conectar(aoMudar: () -> Unit, aoMudarConexao: (Boolean) -> Unit, aoEvento: (String, String?,
Boolean) -> Unit = { _, _, _ -> }) {
93          if (fechado) return
94          val endereco = servidor.replaceFirst("https://", "wss://").replaceFirst("http://", "ws://") +
"/v1/events"
95          socket = cliente.newWebSocket(Request.Builder().url(endereco).build(), object:
WebSocketListener() {
96              override fun onMessage(webSocket: WebSocket, text: String) {
97                  val evento = runCatching { JSONObject(text) }.getOrNull() ?: return
98                  principal.post { if (!fechado) {
99                      aoEvento(evento.optString("type"), evento.optString(if (evento.has("replica"))
"replica" else "via").takeIf { it.isNotBlank() }, evento.has("eventId"))
100                     when (evento.optString("type")) {
101                         "ready", "resync" -> { aoMudarConexao(evento.optBoolean("connected", true));
aoMudar() }
102                         "broker.status" -> aoMudarConexao(evento.optBoolean("connected", false))
103                         else -> aoMudar()
104                     }
105                 } }
106             }
107             override fun onFailure(webSocket: WebSocket, t: Throwable, resposta: Response?) {
reconectar() }
108             override fun onClosed(webSocket: WebSocket, code: Int, reason: String) { reconectar() }
109             private fun reconectar() { principal.post { if (!fechado) { aoMudarConexao(false);
principal.postDelayed({ conectar(aoMudar, aoMudarConexao, aoEvento) }, 2500) } } }
110         })
111     }
```

Fonte: código do projeto ViaAgora (2026).

6.7 Balanceamento no Nginx

No ambiente local, o Nginx alterna os pedidos entre as duas réplicas. Se uma não responde em 2 segundos, ele tenta a outra e deixa a réplica parada de lado por 30 segundos.

Listagem 7 – Configuração do Nginx (nginx.conf, linhas 1 a 24)

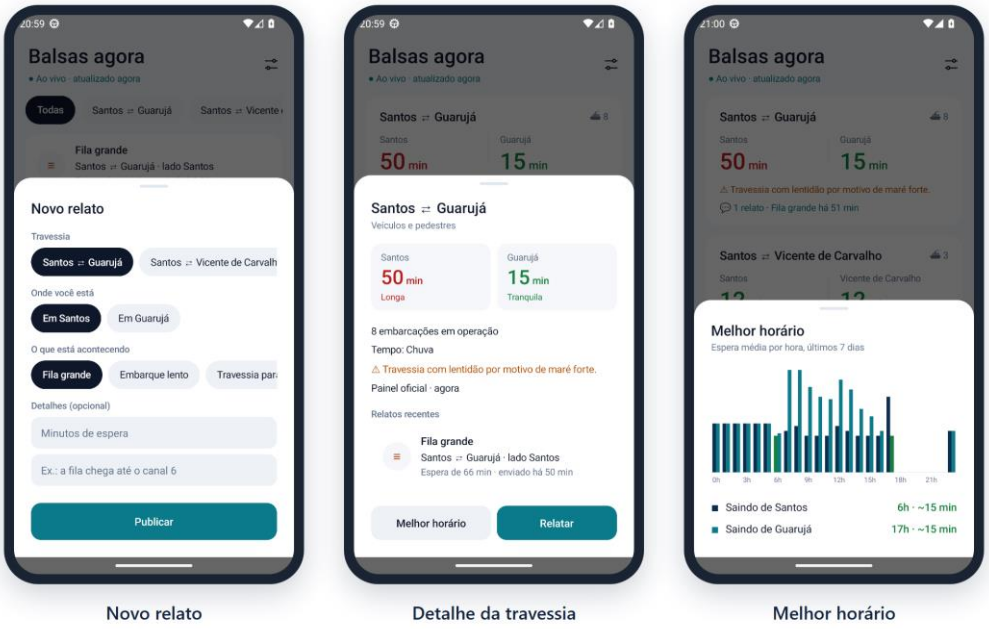
```
1  # Nginx do ambiente local: a mesma ideia da VPS, sem HTTPS.
2  # O site sai da pasta site/ e tudo em /v1 vai para as duas réplicas da API, alternando entre elas.
3  upstream api {
4      zone api 64k;
5      server api:4310 max_fails=1 fail_timeout=30s;
6      server api-2:4310 max_fails=1 fail_timeout=30s;
7  }
8  map $http_upgrade $connection_upgrade { default upgrade; '' close; }
9  server {
10     listen 80;
11     root /usr/share/nginx/html;
12     location /v1/ {
13         proxy_pass http://api;
14         proxy_http_version 1.1;
15         proxy_set_header Upgrade $http_upgrade;
16         proxy_set_header Connection $connection_upgrade;
17         proxy_set_header Host $host;
18         # Réplica parada: desiste em 2 s e tenta a outra, em vez de deixar o celular esperando.
19         proxy_connect_timeout 2s;
20         proxy_read_timeout 75s;
21         proxy_next_upstream error timeout http_502 http_503 http_504;
22     }
23     location = /health { proxy_pass http://api; }
24 }
```

Fonte: código do projeto ViaAgora (2026).

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

O aplicativo foi testado num celular Android e no emulador, conectado ao servidor publicado em sedda.com.br. A Figura 9 mostra o envio de um relato, o detalhe de uma travessia e o melhor horário calculado com o histórico real.

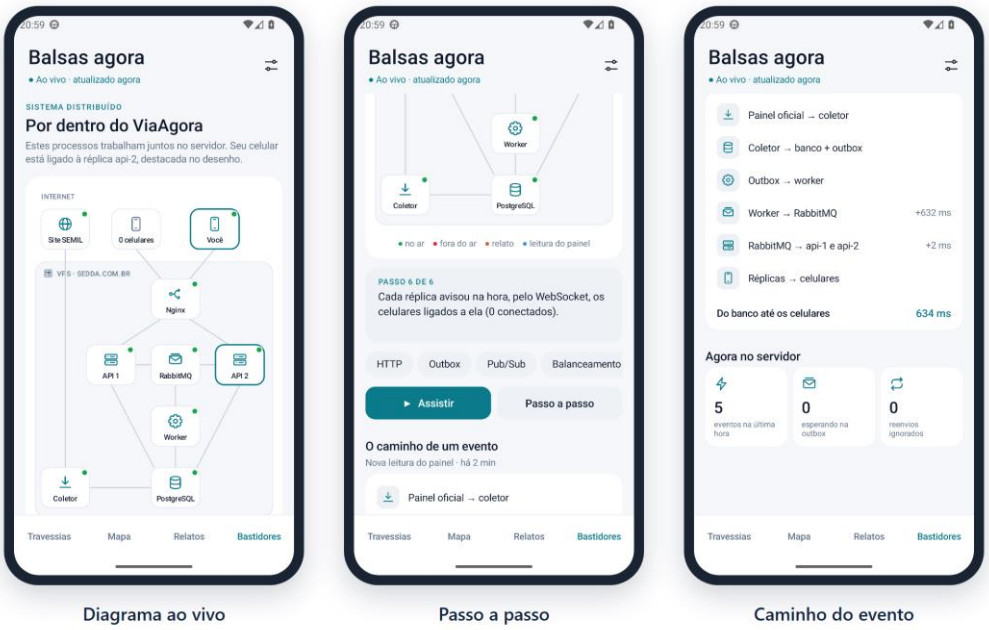
Figura 9 – Novo relato, detalhe da travessia e melhor horário



Fonte: elaborado pelo autor (2026).

A aba **Bastidores** mostra o sistema distribuído funcionando (Figura 10): cada quadro é um processo, com a bolinha verde do sinal de vida; o celular do usuário e a réplica a que está ligado aparecem destacados; e o último evento é animado no desenho, com o tempo medido de cada etapa.

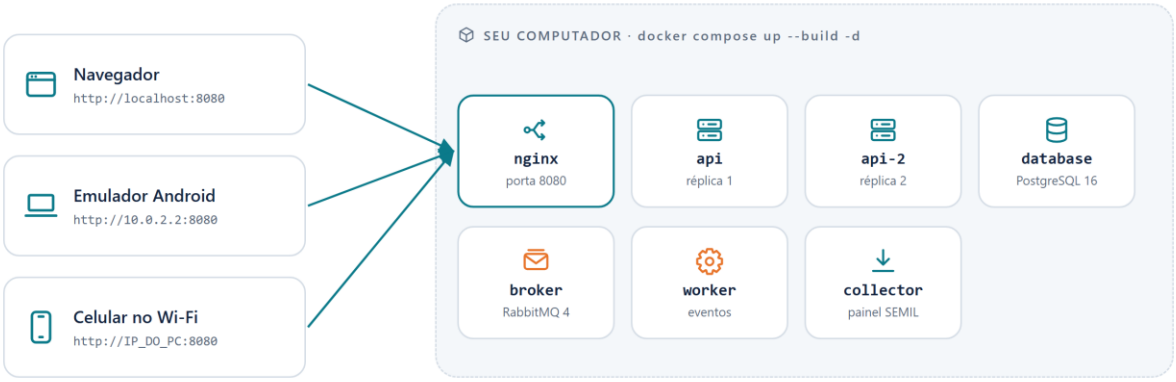
Figura 10 – Aba Bastidores: diagrama ao vivo, passo a passo e caminho do evento



Fonte: elaborado pelo autor (2026).

Para executar localmente, basta clonar o repositório e rodar `docker compose up --build -d`. Em cerca de 30 segundos os sete serviços estão no ar, o site abre em `http://localhost:8080` e o aplicativo se conecta pelo menu Ajustes → Outro servidor (Figura 11).

Figura 11 – Execução local com Docker Compose



Fonte: elaborado pelo autor (2026).

Roteiro da apresentação:

1. Abrir o aplicativo e mostrar as três travessias com os dados oficiais e o mapa.
2. Enviar um relato num celular e mostrar que ele aparece no outro em poucos segundos.

3. Abrir a aba Bastidores e acompanhar o caminho do evento pelo sistema.
4. No ambiente local, parar o RabbitMQ, enviar um relato e mostrar o evento esperando na outbox; religar o broker e ver a entrega.
5. Parar uma réplica da API e mostrar que o Nginx passa tudo para a outra.

8 CONCLUSÃO

O ViaAgora cumpriu o objetivo de levar ao celular, em tempo real, o tempo de espera das balsas da Baixada Santista, combinando o dado oficial com os relatos de quem está na fila. O sistema está publicado, pode ser instalado por qualquer pessoa e roda também no computador de quem quiser estudá-lo.

Do ponto de vista da disciplina, o projeto aplicou os principais conceitos de sistemas distribuídos num cenário real: replicação com balanceamento, comunicação por mensagens no modelo publish/subscribe, consistência entre banco e mensageria com o padrão outbox, idempotência e recuperação de falhas. Os testes derrubando o broker e as réplicas mostraram que nenhum relato se perde e que o serviço continua no ar, que é exatamente a transparência de falha descrita na literatura.

As limitações também ficaram claras. Todos os processos estão numa única VPS, então a queda da máquina derruba o serviço; a fonte oficial não é uma API documentada e pode mudar; e não há moderação dos relatos. Como trabalhos futuros, ficam a distribuição dos processos em mais de um servidor, um banco com réplica de leitura e a moderação colaborativa dos relatos.

BIBLIOGRAFIA

ANDROID DEVELOPERS. **Android Studio** e **documentação do Android**. Google, 2026. Disponível em: https://developer.android.com. Acesso em: 29 out. 2026.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER. **Docker Compose documentation**. Disponível em: https://docs.docker.com/compose/. Acesso em: 29 out. 2026.

FASTIFY. **Fastify documentation**. Disponível em: https://fastify.dev/docs/latest/. Acesso em: 29 out. 2026.

FETTE, Ian; MELNIKOV, Alexey. **RFC 6455**: the WebSocket protocol. IETF, 2011. Disponível em: https://www.rfc-editor.org/rfc/rfc6455. Acesso em: 29 out. 2026.

KLEPPMANN, Martin. **Designing data-intensive applications**. Sebastopol: O'Reilly Media, 2017.

KOTLIN. **Kotlin documentation**. JetBrains, 2026. Disponível em: https://kotlinlang.org/docs/home.html. Acesso em: 29 out. 2026.

MICROSOFT. **Visual Studio Code documentation**. Disponível em: https://code.visualstudio.com/docs. Acesso em: 29 out. 2026.

NGINX. **Using nginx as HTTP load balancer**. Disponível em: https://nginx.org/en/docs/http/load_balancing.html. Acesso em: 29 out. 2026.

OPENSTREETMAP FOUNDATION. **OpenStreetMap**: direitos autorais e licença. Disponível em: https://www.openstreetmap.org/copyright. Acesso em: 29 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 documentation**. Disponível em: https://www.postgresql.org/docs/16/. Acesso em: 29 out. 2026.

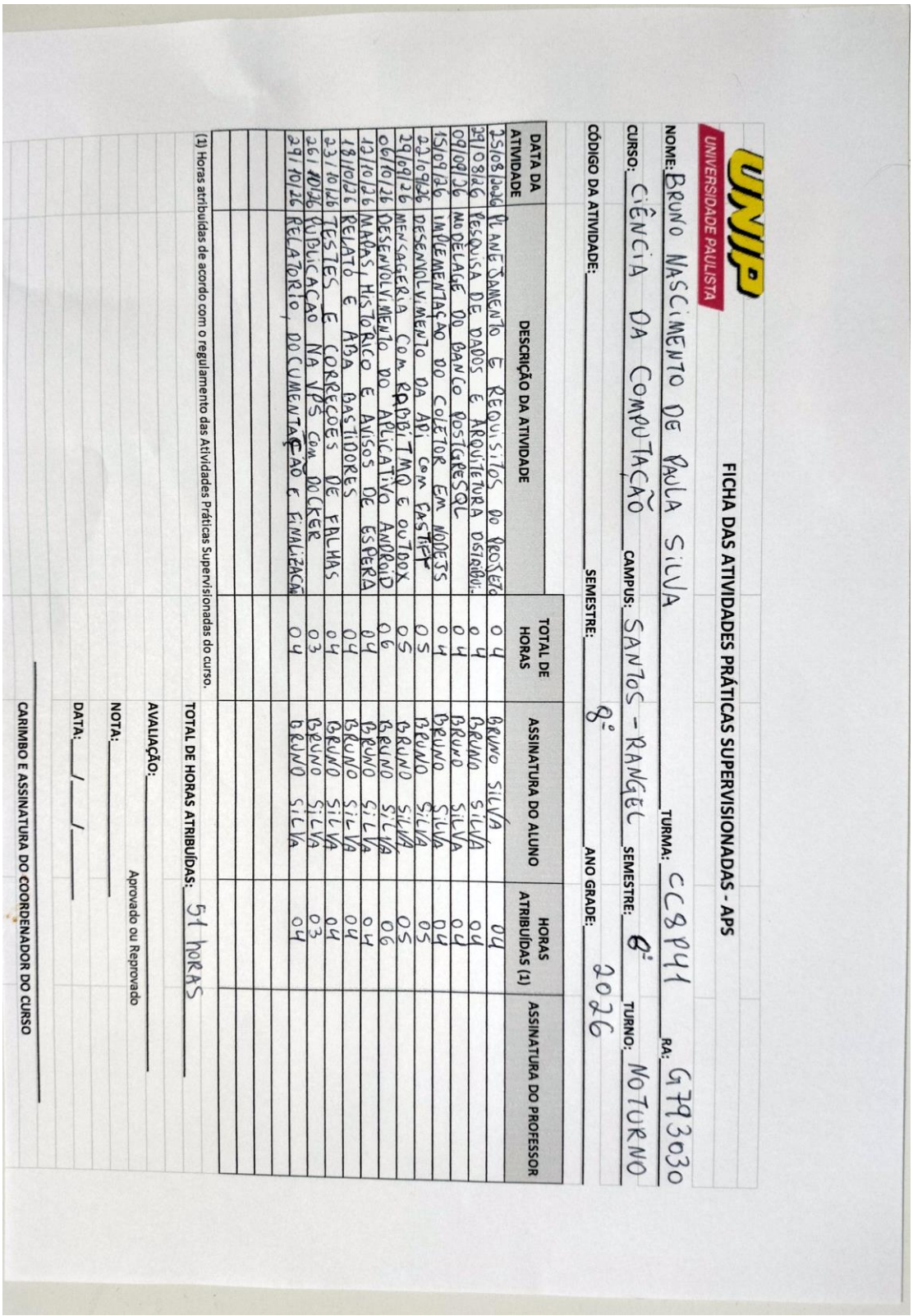
RABBITMQ. **Publish/subscribe** e **Consumer acknowledgements and publisher confirms**. Disponível em: https://www.rabbitmq.com/docs. Acesso em: 29 out. 2026.

RICHARDSON, Chris. **Microservices patterns**. Shelter Island: Manning, 2018.

SÃO PAULO (Estado). Secretaria de Meio Ambiente, Infraestrutura e Logística. **Travessias litorâneas**. Disponível em: https://semil.sp.gov.br/travessias/. Acesso em: 29 out. 2026.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.

FICHA DE ATIVIDADES PRÁTICAS SUPERVISIONADAS



UNIP
UNIVERSIDADE PAULISTA

FICHA DAS ATIVIDADES PRÁTICAS SUPERVISIONADAS - APS

NOME: BRUNO NASCIMENTO DE PAULA SILVA TURMA: CC8P41 RA: G79030

CURSO: CIÊNCIA DA COMPUTAÇÃO CAMPUS: SANTOS - RANGEL SEMESTRE: 8º TURNO: NOTURNO

CÓDIGO DA ATIVIDADE: SEMESTRE: 8º ANO GRADE: 2026

DATA DA ATIVIDADE	DESCRIÇÃO DA ATIVIDADE	TOTAL DE HORAS	ASSINATURA DO ALUNO	HORAS ATRIBUÍDAS (1)	ASSINATURA DO PROFESSOR
25/08/2026	PLANEJAMENTO E REQUISITOS DO PROJETO	04	BRUNO SILVA	04	
29/08/26	PESQUISA DE DADOS E ARQUITETURA DISTRIBUÍ-	04	BRUNO SILVA	04	
08/09/26	MODELAGEM DO BANCO POSTGRESQL	04	BRUNO SILVA	04	
15/09/26	IMPLEMENTAÇÃO DO COLETOR EM NODEJS	04	BRUNO SILVA	04	
22/09/26	DESENVOLVIMENTO DA API COM FASTIFY	05	BRUNO SILVA	05	
29/09/26	MENSAGERIA COM RABBITMQ E OUTBOX	05	BRUNO SILVA	05	
06/10/26	DESENVOLVIMENTO DO APLICATIVO ANDROID	06	BRUNO SILVA	06	
13/10/26	MAPAS, HISTÓRICO E AVISOS DE ESPERA	04	BRUNO SILVA	04	
18/10/26	RELATO E ADA PASSAGEIROS	04	BRUNO SILVA	04	
23/10/26	TESTES E CORREÇÕES DE FALHAS	04	BRUNO SILVA	04	
26/10/26	PUBLICAÇÃO NA VPS COM DOCKER	03	BRUNO SILVA	03	
29/10/26	RELATÓRIO, DOCUMENTAÇÃO E FINALIZAÇÃO	04	BRUNO SILVA	04	

(1) Horas atribuídas de acordo com o regulamento das Atividades Práticas Supervisionadas do curso.

TOTAL DE HORAS ATRIBUÍDAS: 51 HORAS

AVALIAÇÃO: _____

Aprovado ou Reprovado

NOTA: _____

DATA: ___/___/_____

CARIMBO E ASSINATURA DO COORDENADOR DO CURSO